DebugQuest

Complete Bug Report & Solutions Guide

Tech Fest 2026 — React Debugging Challenge

All 10 Bugs — Exact File & Line — Root Cause — Full Fix

Overview

DebugQuest is a React CTF where the application is intentionally sabotaged across 4 levels. The goal is to progress from the Login page all the way to the Completion page by identifying and fixing each bug. This document covers every bug with its exact location, the concept it tests, what you see when it is broken, why it is broken, and the precise code change needed to fix it.

Bug #	File	Bug Name	Concept
1	LoginPage.jsx	Typo in API endpoint URL	Network / REST
2	GameContext.jsx	Missing async/await in fetchSecretKey	Promises / async
3	ProtectedRoute.jsx	Unauthenticated users let through	Route guards
4	ProtectedRoute.jsx	Level comparison operator inverted	Boolean logic
5	App.jsx ↔ ProtectedRoute	requireLevel vs requiredLevel mismatch	Prop naming
6	DashboardPage.jsx	Direct state mutation - modal never opens	React state
7	DashboardPage.jsx	Continue button fires wrong handler	CSS positioning
8	DashboardPage.jsx	Monochrome toggle double-fires, cancels itself	DOM classList
9	DashboardPage.jsx	Speech never triggered	Web Speech API
10a	FeedbackFinalPage.jsx	Invisible div blocks submit button	z-index / stacking
10b	FeedbackFinalPage.jsx	Team name always reversed + palindrome trap	State transform
10c	FeedbackFinalPage.jsx	Admin toggle resets via event bubbling	Event propagation
10d	FeedbackFinalPage.jsx	Upload bar capped at 99% forever	setInterval logic
10e	FeedbackFinalPage.jsx	Success redirects to /not-found	React Router

Level 1 — Login

L1 Authentication Challenge

File: LoginPage.jsx | GameContext.jsx

To pass Level 1 you must successfully send an OTP to your email and verify it. Two separate bugs block this.

B1 Typo in API Endpoint URL

Concept: *Network API calls / REST*

File & Line

client/src/pages/LoginPage.jsx — Line 34

What you see

Clicking "Send OTP" shows an error message or nothing happens. No OTP ever arrives in your email. The network tab in DevTools shows a 404 Not Found for the OTP request.

Root cause

The fetch call on line 34 hits /api/users/send-otpp (double "p" at the end). The server has no route matching this URL, so every OTP request fails with a 404.

Broken code (line 34):

```
const response = await fetch('/api/users/send-otpp', {   // ← double 'p'
```

Fixed code:

```
const response = await fetch('/api/users/send-otp', {    // ← single 'p'
```

How to find it

Open DevTools → Network tab → click Send OTP. You will see a request to /api/users/send-otpp returning 404. Compare the URL to the verify-otp call right below it in the same file.

B2 Missing async/await in fetchSecretKey

Concept: *JavaScript Promises / async-await*

File & Line

client/src/context/GameContext.jsx — Lines 99–107

What you see

Any component that calls fetchSecretKey() and tries to display the result shows "[object Promise]" as text instead of the actual secret value.

Root cause

The function body calls fetch() without await, so "response" holds a raw Promise object, not the actual HTTP response. Calling .json() on an unresolved Promise also returns another Promise instead of the parsed data. Both awaits are missing.

Broken code (lines 99–107):

```
const fetchSecretKey = async () => {
  const response = fetch('/api/secret-key', {   // missing await
    method: 'GET',
  });
  return response.json();                         // missing await
};
```

Fixed code:

```
const fetchSecretKey = async () => {
  const response = await fetch('/api/secret-key', {   // await added
    method: 'GET',
  });
  return await response.json();                         // await added
};
```

Key concept

In JavaScript, fetch() is asynchronous and returns a Promise. Without await you get the Promise object itself, not its resolved value. You must await fetch() to get the Response, and then await .json() to get the parsed body. Both are async operations.

Level 2 — Loading Screen

L2 **The Waiting Game**
File: ProtectedRoute.jsx | App.jsx

Level 2 is the loading screen. It has a 4-minute real timer, but you can skip it by solving the Fibonacci puzzle or pressing S twice quickly. Three bugs are hidden in the route guard that protects this page and the route definitions in App.jsx.

B3 Unauthenticated users are let through the guard

Concept: *React Router — Route Guards / Authentication*

File & Line

client/src/components/ProtectedRoute.jsx — Lines 10–12

What you see

A user who has never logged in can navigate directly to /loading or /dashboard in the browser address bar and see the page. There is no redirect to the login screen.

Root cause

The null-user check on line 10 says: if there is no user, return children or Outlet. This is backwards. When there is no user the guard should BLOCK access, not allow it. The condition is correctly detecting an unauthenticated user but then doing the wrong thing.

Broken code (lines 10–12):

```
// BROKEN: lets unauthenticated users straight through
if (!user) {
  return children ? children : <Outlet />;
}
```

Fixed code:

```
// FIXED: redirect to login when no user exists
if (!user) {
  return <Navigate to="/" replace />;
}
```

Key concept

A route guard (also called an auth guard) is a wrapper component that checks a condition before rendering. If the condition fails it should redirect, not render.

B4 Level comparison operator is inverted

Concept: *Boolean Logic — Comparison operators*

File & Line

client/src/components/ProtectedRoute.jsx — Line 16

What you see

Even after logging in, trying to reach a protected route either blocks you when you should be allowed, or lets you in when you are not at the required level yet. The redirect logic is backwards.

Root cause

Line 16 uses ">" (greater than). This means "block the user if their level is HIGHER than the required level." It should be "<" (less than), meaning "block if their level is LOWER than required."

Broken code (line 16):

```
// BROKEN: blocks users who ARE at or above the required level
if (currentLevel > requiredLevel) {
  return <Navigate to={`/loading`} replace />;
}
```

Fixed code:

```
// FIXED: blocks users who have NOT yet reached the required level
if (currentLevel < requiredLevel) {
  return <Navigate to="/" replace />;
}
```

B5 Prop name mismatch: requireLevel vs requiredLevel

Concept: *React Props — Naming consistency*

Files & Lines

App.jsx line 22 AND ProtectedRoute.jsx line 6

What you see

The /loading route is completely unguarded — anyone can visit it regardless of their level, because the guard never receives the level requirement. Since requiredLevel is undefined, the comparison "currentLevel < undefined" is always false, so the guard always passes.

Root cause

In App.jsx the prop is spelled "requireLevel" (no "d"). In ProtectedRoute.jsx the function destructures "requiredLevel" (with "d"). JavaScript silently ignores unrecognized props, so the guard receives undefined for its level check.

Broken code:

```
// App.jsx line 22
<Route element={<ProtectedRoute requireLevel={2} />}>    // ← missing "d"

// Also broken: /feedback-final passes a string instead of a number
<ProtectedRoute requiredLevel="level-4">                 // ← should be {4}
```

Fixed code:

```
// App.jsx line 22
<Route element={<ProtectedRoute requiredLevel={2} />}>   // ← "d" added

// Also fix /feedback-final
<ProtectedRoute requiredLevel={4}>                       // ← number not string
```

Key concept

React props are just JavaScript object keys. A typo in the prop name means the component receives undefined. Always double-check that the prop name you pass matches exactly what the component destructures in its function signature.

Level 3 — Dashboard

L3 Dashboard Chaos

File: DashboardPage.jsx

The Dashboard has four independent bugs. Each one breaks a different subsystem. Solving the Fibonacci puzzle (answer: 13-21-34, positions 7, 8, 9 of the sequence starting at 1,1,2,3,5,8,13,21,34) gets you to this level. Alternatively press S twice quickly to skip the loading screen.

B6 Direct state mutation — Decrypt modal never opens

Concept: *React State — Immutability*

File & Line

client/src/pages/DashboardPage.jsx — Line 120

What you see

Clicking the "Decrypt Secret" button does nothing. The modal panel that should reveal the decrypted payload never appears, even though the panel code is right there in the JSX.

Root cause

Line 120 directly mutates the state object: "decryptState.isVisible = true". In React, state is read-only. Mutating it directly does NOT tell React that anything changed, so React never schedules a re-render, and the modal stays hidden. The data changes in memory, but the screen never updates.

Broken code (line 120):

```
const handleDecryptClick = () => {
  decryptState.isVisible = true;   // WRONG: direct mutation, React ignores this
};
```

Fixed code:

```
const handleDecryptClick = () => {
  // CORRECT: use the setter function with spread to preserve other fields
  setDecryptState(prev => ({ ...prev, isVisible: true }));
};
```

Key concept

NEVER mutate state directly in React. Always use the setter returned by useState. The setter triggers a re-render with the new value. When state is an object, spread the previous state (...prev) and override only the fields you want to change.

B7 "Continue" button fires wrong handler — CSS positioning trap

Concept: *CSS — position:absolute / z-index*

File & Line

client/src/pages/DashboardPage.jsx — Lines 252–279

What you see

Clicking "Continue to Final Feedback →" makes the button teleport to a random position on screen instead of navigating to the next level. The button is literally running away from you.

Root cause

There are TWO overlapping elements. The visible button (with onClick=moveButton) makes the button jump. The real navigation div (with onClick=handleContinue) is positioned at top:-9999px, left:-9999px, which places it completely off-screen. So every click hits the fake button, not the real one.

Broken code (lines 273–279):

```
<div
  onClick={handleContinue}
  style={{
    position: 'absolute',
    top: '-9999px',     // ← WAY off screen, unreachable
    left: '-9999px',
    width: '100%',
    height: '100%',
    zIndex: 20,         // ← correct z-index, just wrong position
  }}
/>
```

Fixed code:

```
<div
  onClick={handleContinue}
  style={{
    position: 'absolute',
    top: 0,             // ← now covers the entire container
    left: 0,
    width: '100%',
    height: '100%',
    zIndex: 20,         // ← sits above the fake button (zIndex: 10)
    cursor: 'pointer',
  }}
/>
```

Alternative fix

You can also delete the fake button and its moveButton handler entirely, and change the real div to display normally (remove position:absolute and use a regular visible button with onClick={handleContinue}).

B8 Monochrome toggle double-fires and cancels itself

Concept: *DOM API — classList.toggle*

File & Line

client/src/pages/DashboardPage.jsx — Lines 50–51

What you see

Clicking the B/W toggle in the header appears to do nothing visually. The isMonochromeMode React state does toggle (which you could verify with DevTools), but the actual body class that applies the visual change never changes.

Root cause

Lines 50–51 call document.body.classList.toggle('light-mode') TWICE in a row. The first call adds the class. The second call immediately removes it. The net result is zero — the class is back where it started. The React state variable updates correctly but the DOM class change cancels itself out.

Broken code (lines 50–51):

```
const breakMonochromeMode = () => {
  setIsMonochromeMode(prev => !prev);
  document.body.classList.toggle('light-mode');  // toggles ON
  document.body.classList.toggle('light-mode');  // immediately toggles back OFF
};
```

Fixed code:

```
const breakMonochromeMode = () => {
  setIsMonochromeMode(prev => !prev);
  document.body.classList.toggle('light-mode');  // ← single call only
};
```

B9 Speech synthesis utterance created but never spoken

Concept: *Web Speech API — speechSynthesis.speak()*

File & Line

client/src/pages/DashboardPage.jsx — Lines 54–61

What you see

Clicking "Read Aloud" shows the status message "Synthesizing speech... wait, no sound?" but no audio ever plays. The browser never speaks.

Root cause

The code correctly creates a SpeechSynthesisUtterance object and sets its volume and rate, but then forgets to actually submit it to the browser's speech engine. The speak() call is missing. Creating the utterance object is just setup — nothing plays until you call window.speechSynthesis.speak(utterance).

Broken code (lines 54–61):

```
const handleReadAloudClick = () => {
  const utterance = new SpeechSynthesisUtterance(outroParagraph);
  utterance.volume = 1;
  utterance.rate = 1;
  // Missing: window.speechSynthesis.speak(utterance);
  setNarrationStatus('Synthesizing speech... wait, no sound?');
};
```

Fixed code:

```
const handleReadAloudClick = () => {
  const utterance = new SpeechSynthesisUtterance(outroParagraph);
  utterance.volume = 1;
  utterance.rate = 1;
  window.speechSynthesis.speak(utterance);   // ← this line was missing
  setNarrationStatus('Speaking...');
```

```
};
```

Level 4 — Final Feedback Form

L4

The Final Boss

File: FeedbackFinalPage.jsx

Level 4 is the most complex level. The feedback form has five separate bugs layered on top of each other. You must fix ALL of them before the form will submit successfully and redirect to the completion screen. The form also has special rules you must follow for input values.

Form Rules (these are intentional, not bugs):

- Email must end with exactly "@Gmail" (capital G) — e.g. yourname@Gmail
- Date of Birth must be today or a future date (double-click the DOB label to unlock past dates)
- Phone counter must not end in 2 or 4 (double-click the Phone label to enable step-1 precision mode)
- Message field only accepts binary: digits 0, 1, and spaces only

10a

Invisible div blocks the submit button (z-index trap)

Concept: *CSS — z-index / stacking context*

File & Lines

client/src/pages/FeedbackFinalPage.jsx — Lines 330–345

What you see

The submit button is visible and looks clickable, but clicking it shows "ERR_BLOCKED: An invisible force prevents submission." and the form never submits. This happens even when all fields are correctly filled.

Root cause

There is an absolutely-positioned div covering the entire submit button area with opacity:0 (fully transparent) and zIndex:9999. This invisible layer intercepts every click before it reaches the real button underneath (which only has zIndex:1). The invisible div shows the error message in its own onClick instead of letting the click through.

Broken code (lines 332–345):

```
<div style={{ position: 'relative' }}>
  {/* TRAP: this invisible div is above the button and eats all clicks */}
  <div
    style={{
      position: 'absolute',
      top: 0, left: 0, right: 0, bottom: 0,
      zIndex: 9999,          // way above the button
      opacity: 0,            // completely invisible
      cursor: 'not-allowed',
    }}
    onClick={(e) => {
      e.preventDefault();
      setSubmitStatus("ERR_BLOCKED: An invisible force...");
```

```
    }}
  />
  <button type="submit" style={{ zIndex: 1 }}>Submit</button>
</div>
```

Fixed code — delete the entire invisible div:

```
<div style={{ position: 'relative' }}>
  {/* Remove the invisible trap div entirely */}
  <button type="submit" className="cyber-btn"
    style={{ width: '100%' }}
    disabled={isSubmitting}
  >
    {isSubmitting ? 'Submitting...' : 'Submit Final Feedback'}
  </button>
</div>
```

10b Team name always reversed + palindrome trap

Concept: *React State — Input transforms*

File & Lines

client/src/pages/FeedbackFinalPage.jsx — Lines 37–39 (updateField) and Lines 115–117 (handleSubmit)

What you see

When typing your team name it appears reversed on screen (the CSS direction:rtl makes it mirror). Then on submit you get "Security Check: Name string proxy invalid (must be a palindrome to save)" — an impossible requirement for any normal name.

Root cause

Two traps working together. First, updateField() reverses every character you type (line 38), so "Team" is stored as "maeT". Second, handleSubmit() (line 115) checks if the stored name equals its own reverse — i.e. it must be a palindrome. Since no normal team name is a palindrome, and the value is already reversed in state, this check can never pass. The input also has CSS direction:rtl which visually mirrors text.

Broken code:

```
// In updateField() — lines 36-39
if (field === 'teamName') {
  finalValue = value.split('').reverse().join('');  // reverses every keystroke
}

// In handleSubmit() — lines 115-117
if (formState.teamName !== formState.teamName.split('').reverse().join('')) {
  setSubmitStatus('...must be a palindrome to save.');
  return;
}

// In JSX — the input element
<input style={{ unicodeBidi: 'bidi-override', direction: 'rtl' }} />
```

Fixed code — remove all three traps:

```
// In updateField() — remove the reversal entirely
const updateField = (field, value) => {
  setFormState(prev => ({ ...prev, [field]: value }));  // just store value
directly
};

// In handleSubmit() — delete the palindrome check block entirely

// In JSX — remove the mirroring CSS
<input placeholder="Team identifier" />   // no direction/unicodeBidi styles
```

10c Admin toggle immediately resets via event bubbling

Concept: *JavaScript — Event bubbling / stopPropagation*

File & Lines

client/src/pages/FeedbackFinalPage.jsx — Lines 218–232

What you see

Clicking the "REQUEST ADMIN ACCESS" toggle flashes "ADMIN ACCESS GRANTED" for a split second and then immediately resets back to "REQUEST ADMIN ACCESS". You can never hold the admin toggle in the ON state.

Root cause

Event bubbling. When you click the inner toggle div, its onClick fires first and sets isAdmin to true. Then the click event "bubbles up" to the parent container div, whose onClick fires next and sets isAdmin back to false. Both handlers fire for a single click, in that order, so the toggle always ends at false.

Broken code (lines 218–232):

```
<div onClick={() => setIsAdmin(false)}>      {/* parent resets to false */}
  <div
    className="toggle-broken"
    onClick={(e) => {
      setIsAdmin(!isAdmin);                  {/* child sets to true... */}
    }}                                       {/* ...then parent fires: false! */}
  >
    ...
  </div>
</div>
```

Fixed code — stop the event from bubbling up:

```
<div onClick={() => setIsAdmin(false)}>
  <div
    className="toggle-broken"
    onClick={(e) => {
      e.stopPropagation();    // ← stops event from reaching parent div
      setIsAdmin(!isAdmin);
    }}
  >
    ...
```

```
  </div>
</div>
```

Key concept

In the browser, click events bubble upward through every parent element. e.stopPropagation() stops this — the event is handled by the current element and goes no further. Alternatively, you could remove the onClick from the parent div entirely.

10d Upload progress bar stuck forever at 99%

Concept: *JavaScript — setInterval / off-by-one*

File & Lines

client/src/pages/FeedbackFinalPage.jsx — Lines 77–80

What you see

Clicking "Upload Avatar" starts a progress bar that fills up quickly, but gets permanently stuck at 99% showing "Network Congestion: Stuck at Silk Board Junction." The form then blocks submission saying the upload is not complete.

Root cause

The setInterval callback (line 77) checks "if next >= 99" and returns 99, capping the value forever at 99. The interval is cleared but the value never reaches 100. The handleSubmit function on line 108 checks "uploadProgress < 100" before allowing submission, so 99% permanently blocks the form.

Broken code (lines 77–80):

```
if (next >= 99) {
  clearInterval(interval);
  return 99;    // ← stuck at 99, never reaches 100
}
```

Fixed code:

```
if (next >= 100) {
  clearInterval(interval);
  return 100;   // ← now actually completes
}
```

10e Final success redirects to /not-found instead of /complete

Concept: *React Router — navigate()*

File & Line

client/src/pages/FeedbackFinalPage.jsx — Line 149

What you see

After successfully filling out the form and clicking Submit, the app redirects you to the 404 Not Found page instead of the Completion/Leaderboard screen. This is the cruelest trap — placed right at the end, after you have fixed everything else.

Root cause

The navigate() call on line 149 sends the user to "/not-found" instead of "/complete". The completion data (totalTime) is correctly computed and passed, but it ends up on the wrong page.

Broken code (line 149):

```
// After successful submission:
navigate('/not-found', { state: { totalTime } });   // ← wrong route!
```

Fixed code:

```
// After successful submission:
navigate('/complete', { state: { totalTime } });    // ← correct route
```

Quick Reference Summary

Use this page as a fast-check during your debugging session. Each fix is the minimal change needed.

#	Location	The One-Line Fix	Concept
1	`LoginPage.jsx:34`	send-otpp → send-otp	Typo
2	`GameContext.jsx:103`	Add await before fetch() and .json()	async/await
3	`ProtectedRoute.jsx:11`	Return <Navigate to="/" /> when !user	Auth guard
4	`ProtectedRoute.jsx:16`	Change > to < in level comparison	Boolean logic
5	`App.jsx:22`	requireLevel → requiredLevel, "level-4" → {4}	Prop name
6	`DashboardPage.jsx:120`	Use setDecryptState() not direct mutation	State immutability
7	`DashboardPage.jsx:276`	-9999px → 0 for top/left on handleContinue div	CSS position
8	`DashboardPage.jsx:51`	Remove the second classList.toggle() call	DOM API
9	`DashboardPage.jsx:59`	Add window.speechSynthesis.speak(utterance)	Web Speech API
10a	`FeedbackFinalPage.jsx:332`	Delete the invisible overlay div entirely	z-index
10b	`FeedbackFinalPage.jsx:38`	Remove .reverse().join() from updateField	Input transform
10c	`FeedbackFinalPage.jsx:229`	Add e.stopPropagation() in toggle onClick	Event bubbling
10d	`FeedbackFinalPage.jsx:77`	Change >= 99 → >= 100, return 99 → 100	setInterval
10e	`FeedbackFinalPage.jsx:149`	/not-found → /complete in navigate()	React Router